

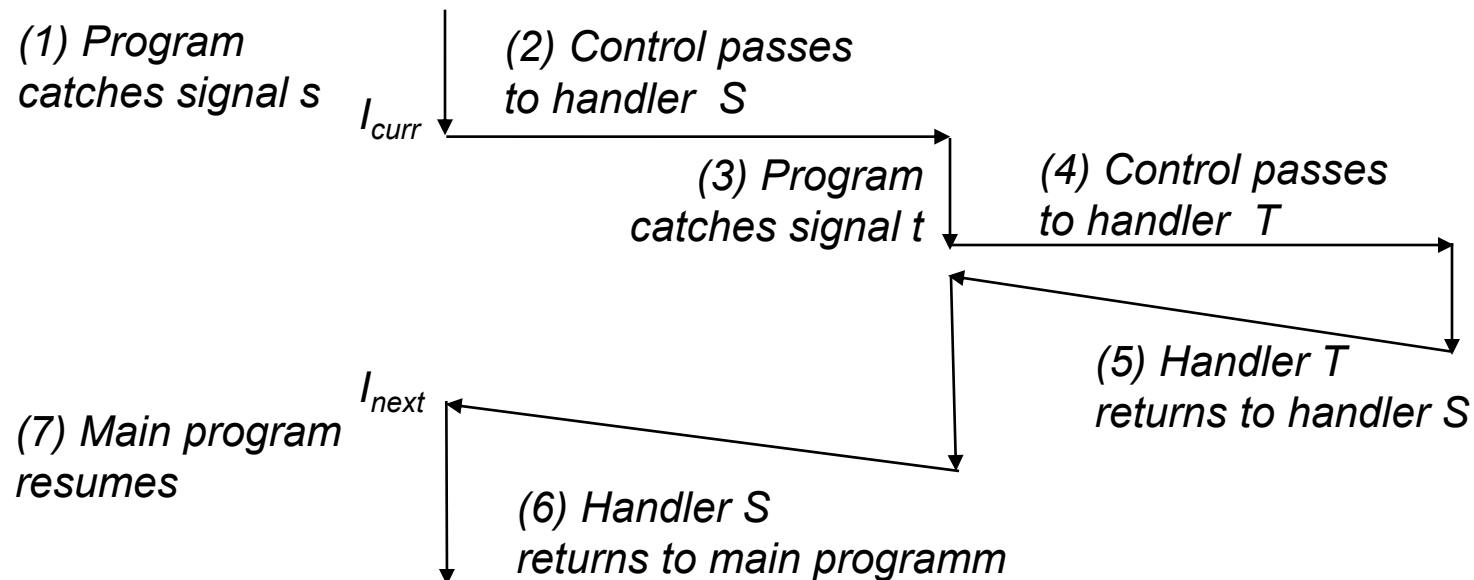
Exceptional Control Flow

Outline

- Suggested reading Signals
 - Writing Signal Handlers
 - Nasty Concurrency Bugs
 - Explicitly waiting for signals
- Suggested reading 8.5

Safe Signal Handling

Main Program and Signal Handlers are Concurrent



$$s \neq t$$

Concurrent Bugs

- Concurrent bugs
 - Tricky and subtle
 - A program works correctly most the time
 - When it fails, it fails in unpredictable and unrepeatable ways that are horrendously difficult to debug

Guidelines

- GO keep handlers as simple as possible
 - Might simply set a global flag and return immediately
 - all processing associated with the receipt of the signal is performed by the main program
 - which periodically checks (and reset) the flag

Guidelines

- *G1* call only asyn-signal-safe functions in your handler
 - Reentrant or cannot be interrupted
 - `printf`, `sprintf`, `malloc`, `exit` are not safe

_Exit	fexecve	poll	sigqueue
_exit	fork	posix_trace_event	sigset
abort	fstat	pselect	sigsuspend
accept	fstatat	raise	sleep
access	fsync	read	socketatmark
aio_error	ftruncate	readlink	socket
aio_return	futimens	readlinkat	socketpair
aio_suspend	getegid	recv	stat
alarm	geteuid	recvfrom	symlink
bind	getgid	recvmsg	symlinkat
cfgetispeed	getgroups	rename	tcdrain
cfgetospeed	getpeername	renameat	tcflow
cfsetispeed	getpgrp	rmdir	tcflush
cfsetospeed	getpid	select	tcgetattr
chdir	getppid	sem_post	tcgetpgrp
chmod	getsockname	send	tcsendbreak
chown	getsockopt	sendmsg	tcsetattr
clock_gettime	getuid	sendto	tcsetpgrp
close	kill	setgid	time
connect	link	setpgid	timer_getoverrun
creat	linkat	setsid	timer_gettime
dup	listen	setsockopt	timer_settime
dup2	lseek	setuid	times
execl	lstat	shutdown	umask
execle	mkdir	sigaction	uname
execv	mkdirat	sigaddset	unlink
execve	mkfifo	sigdelset	unlinkat
faccessat	mkfifoat	sigemptyset	utime
fchmod	mknod	sigfillset	utimensat
fchmodat	mknodat	sigismember	utimes
fchown	open	signal	wait
fchownat	openat	sigpause	waitpid
fcntl	pause	sigpending	write
fdatasync	pipe	sigprocmask	

Safe I/O Package

```
#include <csapp.h>
```

```
ssize_t sio_putl(long v);
```

```
ssize_t sio_puts(char s[]);
```

Returns: number of bytes transferred if OK, -1 on error

```
void sio_error(long v);
```

Returns: nothing

- `_exit` is an `asyn_signal_safe` variant of `exit`

Safe I/O Package

```
ssize_t sio_puts(char s[]) /* Put string */
{
    return write(STDOUT_FILENO, s, sio_strlen(s));
}

ssize_t sio_putl(long v) /* Put long */
{
    char s[128];
    sio_ltoa(v, s, 10); /* Based on K&R itoa() */
    return sio_puts(s);
}

void sio_error(char s[]) /* Put error message and exit */
{
    sio_puts(s);
    _exit(1);
}
```

Guidelines

- *G2* Save and restore errno
 - Many `async_signal_safe` functions set `errno` when they return with an error
 - Calling such functions inside a handler might interfere with other parts of the program that rely on `errno`

Correct Signal Handling

```
14 int main()
15 {
16     int i, n;  char buf[MAXBUF];
17     if (signal(SIGCHLD, handler1) == SIG_ERR)
18         unix_error("signal error");
19
20     /* parent creates children */
21     for (i = 0; i < 3; i++) {
22         if (Fork() == 0) {
23             printf("Hello from child %d\n", (int)getpid());
25             exit(0);
26         }
27     }

28     /* parent waits for term. input and then processes it */
29     if ((n = read(STDIN_FILENO, buf, sizeof(buf))) < 0)
30         unix_error("read");
31
32     printf("Parent processing input\n");
33     while (1) ;
34
35     exit(0);
36 }
```

```
1  #include "csapp.h"
2
3  void handler1(int sig)
4  {
5      int olderrno = errno;
6
7      if ((pid = waitpid(-1, NULL, 0)) < 0)
8          sio_error("waitpid error");
9      sio_puts("Handler reaped child\n");
10     sleep(1);
11     errno = olderrno ;
12 }
13
```

```
linux> ./signal1
Hello from child 14073
Hello from child 14074
Hello from child 14075
Handler reaped child
Handler reaped child
<cr>
Parent processing input
<ctrl-z>
Suspended
```

```
linux> ps t
PID      TTY      STAT    TIME COMMAND
...
14072 pts/3  T        0:02  ./signal1
14075 pts/3  Z        0:00  [signal1] <defunct>
14076 pts/3  R        0:00  ps t
```

Signal Handling Issues

- Pending signals can be blocked
 - Unix signal handlers typically block pending signals of the type currently being processed by the handler
- Pending signals are not queued
 - There can be at most one pending signal of any particular type
 - then the second same type of pending signal is simply discarded


```
1 void handler2(int sig)
2 {
3     int olderrno = errno;
4
5     while ((pid = waitpid(-1, NULL, 0)) > 0) {
6         sio_puts("Handler reaped child\n");
7     }
8     if (errno != ECHILD)
9         sio_error("waitpid error");
10    sleep(1);
11    errno = olderrno ;
12 }
```

```
linux> ./signal2
Hello from child 10378
Hello from child 10379
Hello from child 10380
Handler reaped child
Handler reaped child
Handler reaped child
<cr>
Parent processing input
```

Portable Signal Handling

Different Signal-handling Semantics

- Slow system calls
 - read, write, wait, accept
 - can potentially block the process for a long period of time
- Some older Unix systems
 - restore the action for signal k to its default after signal k has been caught by a handler
 - return to the user immediately after a slow system call is interrupted (do not return to slow system call itself)

```
solaris> ./signal2
```

```
Hello from child 18906
```

```
Hello from child 18907
```

```
Hello from child 18908
```

```
Handler reaped child
```

```
Handler reaped child
```

```
Handler reaped child
```

```
read: Interrupted system call
```

```
15 int main()
16 {
17     int i, n;  char buf[MAXBUF];
18     if (signal(SIGCHLD, handler2) == SIG_ERR)
19         unix_error("signal error");
20
21     /* parent creates children */
22     for (i = 0; i < 3; i++) {
23         if (Fork() == 0) {
24             printf("Hello from child %d\n", (int)getpid());
25             Sleep(1);
26             exit(0);
27         }
28     }
29     /* Manually restart the read if it is interrupted */
30     while ((n = read(STDIN_FILENO, buf, sizeof(buf))) < 0)
31         if (errno != EINTR)
32             unix_error("read");
33
34     printf("Parent processing input\n");
35     while (1) ;
36     exit(0);
37 }
```

```
solaris> ./signal3
```

```
Hello from child 19571
```

```
Hello from child 19572
```

```
Hello from child 19573
```

```
Handler reaped child 19571
```

```
Handler reaped child 19572
```

```
Handler reaped child 19573
```

```
<cr>
```

```
Parent processing input
```

sigaction Function

```
#include <signal.h>

int sigaction(int signum,
              struct sigaction *act,
              struct sigaction *oldact);
                      returns: 0 if OK, -1 on error

struct sigaction {
    void (*sa_handler)();           /* addr of signal handler,
                                     or SIG_IGN, or SIG_DFL */
    sigset_t sa_mask; /* additional signals to block */
    int sa_flags;       /* signal options */
};
```



```
1  handler_t *Signal(int signum, handler_t *handler)
2  {
3      struct sigaction action, old_action;
4
5      action.sa_handler = handler;
6      /* block sigs of type being handled */
7      sigemptyset(&action.sa_mask);
8      /* restart syscalls if possible */
9      action.sa_flags = SA_RESTART;
10
11     if (sigaction(signum, &action, &old_action) < 0)
12         unix_error("Signal error");
13     return (old_action.sa_handler);
14 }
```

Signal Function

- Only signals of the type currently being processed by the handler are **blocked**
- As with all signal implementations, signals **are not queued**
- Interrupted system calls are **automatically restarted** whenever possible
- Once the signal handler is installed, it remains installed until `Signal` is called with a handler argument of either `SIG_IGN` or `SIG_DEF`

A Nasty Concurrency Bug

```
1  /* WARNING: This code is buggy */
2  void handler(int sig)
3  {
4      int olderrno = errno ;
5      pid_t pid ;
6
7      while ( (pid = waitpid(-1, NULL, 0) > 0)) > 0) { /*reap a zombie child */
8          deletejob(pid); /*delete the child from the job list*/
9      }
10     if (errno != ECHILD)
11         sio_error("waitpid error") ;
12     errno = olderrno ;
13 }
```

```
14 int main(int argc, char **argv)
15 {
16     int pid;
17
18     Signal(SIGCHLD, handler);
19     initjobs(); /* initialize the job list */
20
21     while(1) {
22         if ((pid = Fork()) == 0) { /*Child process */
23             Execve("/bin/ls", argv, NULL);
24         }
25         addjob(pid) ; /* Parent process adds the child to the job list */
26     }
27     exit(0)
28 }
```

Guidelines

- G3 Protect accesses to shared global data structures by blocking all signals

```

19 int main(int argc, char **argv)
20 {
21     int pid;
22     sigset_t mask_all, prev_all ;
23
24     Sigfillset(&mask_all);
25     Signal(SIGCHLD, handler);
26     initjobs(); /* initialize the job list */
27
28     while(1) {
29         if ((pid = Fork()) == 0) { /*Child process */
30             Execve("/bin/ls", argv, NULL);
31         }
32         /* parent process */
33         addjob(pid) ; /* Add the child to the job list */
34         Sigprocmask(SIG_SETMASK, &prev_all, NULL);
35     }
36     exit(0)
37 }

```

```

1  /* WARNING: This code is still buggy */
2  void handler(int sig)
3  {
4      int olderrno = errno ;
5      sigset_t mask_all, prev_all;
6      pid_t pid ;
7
8      Sigfillset(&mask_all) ;
9      /*reap a zombie child */
10     while ((pid = waitpid(-1, NULL, 0) > 0)) > 0) {
11         SigProcmask(SIG_BLOCK, &mask_all, &prev_all);
12         deletejob(pid); /*delete the child from the job list*/
13         SigProcmask(SIG_SETMASK, &prev_all, NULL);
14     }
15     if (errno != ECHILD)
16         sio_error("waitpid error") ;
17     errno = olderrno ;
18 }

```


Nasty Concurrency Bugs

- Incorrectly sequence of events
 - Parent executes `fork` and kernel schedules the newly created child
 - Child terminates and kernel delivers a `SIGCHLD` signal to parent
 - Parent receives the `SIGCHLD` signal and runs the signal handler
 - Signal handler calls `deletejob`
 - Parent return from `fork` and calls `addjob`

```
18 int main(int argc, char **argv)
19 {
20     int pid;
21     sigset_t mask_all, mask_one, prev_one ;
22
23     Sigfillset(&mask_all);
24     Sigemptyset(&mask_one);
25     Sigaddset(&mask_one, SIGCHLD);
26     Signal(SIGCHLD, handler);
27     initjobs(); /* initialize the job list */
28
29     while(1) {
30         Sigprocmask(SIG_BLOCK, &mask_one, &prev_one)/* Block SIGCHLD
31         if ((pid = Fork()) == 0) { /*Child process */
32             Sigprocmask(SIG_SETMASK, &prev_one, NULL);
33             Execve("/bin/ls", argv, NULL); Unblock SIGCHLD
34     }
```

```
/* parent process */
```

```
35      Sigprocmask(SIG_BLOCK, &mask_all, NULL);
```

```
36      addjob(pid) ; /* Add the child to the job list */
```

```
37      Sigprocmask(SIG_SETMASK, &prev_one, NULL);
```

```
38  }
```

```
38      exit(0)
```

```
40 }
```

Explicitly Waiting for Signals

Guidelines

- G4 Declare global variables with `volatile`
 - Not to put the global into a register
- G5 Declare flags with `sig_atomic_t`
 - atomic read/write is guaranteed among the main program and signal handlers
 - Compound operations are not guaranteed to be atomic such as `flag++` or `flag+=10`

```
1  #include "csapp.h"
2
3  volatile sig_atomic_t pid;
4
5  void sigchld_handler(int s)
6  {
7      int olderrno = errno;
8      pid = waitpid(-1, NULL, 0);
9      errno = olderrno;
10 }
11
12 void sigint_handler(int s)
13 {
14 }
15
```

```
16 int main(int argc, char **argv)
17 {
18     sigset_t mask, prev;
19
20     Signal(SIGCHLD, sigchld_handler);
21     Signal(SIGINT, sigint_handler);
22     Sigemptyset(&mask);
23     Sigaddset(&mask, SIGCHLD);
24
25     while (1) {
26         /* Block SIGCHLD */
27         Sigprocmask(SIG_BLOCK, &mask, &prev);
28         if (Fork() == 0) /* Child */
29             exit(0);
```

```
30      /* Parent */
31      pid = 0;
32      /* Unblock SIGCHLD */
33      Sigprocmask(SIG_SETMASK, &prev, NULL);
34
35      /* Wait for SIGCHLD to be received */
36      while (!pid)
37          ;
38
39      /* Do some work after receiving SIGCHLD */
40      printf(".");
41  }
42  exit(0);
}
```


Eliminate the Spin Loop

```
while (!pid)
    ;
```

- The spin loop is wasteful for CPU resources

Eliminate the Spin Loop

- Inserting a pause in the body of the spin loop:

```
while (!pid)          /* Race! */  
    pause();
```

- Still need a loop because pause might be interrupted by the receipt of one or more SIGINT signals
- This code has a serious **race condition**:
 - if the SIGCHLD is received after the while test but before the pause
 - the pause will sleep forever

Eliminate the Spin Loop

- Replace the pause with sleep:

```
while (!pid) /* Too slow! */  
    sleep(1);
```

- This code is **too slow**
 - If the signal is received after the while and before the sleep
 - the program must wait a (relatively) long time before it can check the loop termination condition again

Eliminate the Spin Loop

- Using a higher-resolution sleep function such as nanosleep isn't acceptable
 - because there is no good rule for determining the sleep interval
- Make it too small
 - the loop is too wasteful
- Make it too high
 - the program is too slow

signalsuspend Function

```
#include <signal.h>
```

```
int sigsuspend(const sigset_t *mask);
```

returns: -1

- temporarily replaces the current blocked set with mask
- suspends the process until the receipt of a signal whose action is either to run a handler or to terminate the process
- If the action is to terminate, then the process terminates without returning from it.
- If the action is to run a handler, then it returns after the handler returns, restoring the blocked set to its state when it was called.

Eliminate the Spin Loop

- The sigsuspend function is equivalent to an **atomic** version of the following:
 - 1 *sigprocmask(SIG_SETMASK, &mask, &prev);*
 - 2 *pause();*
 - 3 *sigprocmask(SIG_SETMASK, &prev, NULL);*
- The **atomic** (uninterruptible) property guarantees that
 - the calls to sigprocmask (line 1) and pause (line 2) occur together, without being interrupted

```
1 #include "csapp.h"
2
3 volatile sig_atomic_t pid;
4
5 void sigchld_handler(int s)
6 {
7     int olderrno = errno;
8     pid = waitpid(-1, NULL, 0);
9     errno = olderrno;
10 }
11
12 void sigint_handler(int s)
13 {
14 }
15
```

```
16 int main(int argc, char **argv)
17 {
18     sigset_t mask, prev;
19
20     Signal(SIGCHLD, sigchld_handler);
21     Signal(SIGINT, sigint_handler);
22     Sigemptyset(&mask);
23     Sigaddset(&mask, SIGCHLD);
24
25     while (1) {
26         /* Block SIGCHLD */
27         Sigprocmask(SIG_BLOCK, &mask, &prev);
28         if (Fork() == 0) /* Child */
29             exit(0);
```



```
30      /* Wait for SIGCHLD to be received */
31      pid = 0;
32      while (!pid)
33          sigsuspend(&prev) ;
34
35      /* Optionally unblock SIGCHLD */
36      Sigprocmask(SIG_SETMASK, &prev, NULL) ;
37
38      /* Do some work after receiving SIGCHLD */
39      printf(".");
40  }
41  exit(0) ;
42 }
```

Exceptional Control Flow VI

Outline

- Nonlocal Jump
- Suggested reading 8.6

Nonlocal Jumps

Nonlocal Jumps

- Nonlocal Jumps
 - transfers control directly from one function to another currently executing function
 - without having to go through the normal call-and-return sequence
 - C provides a form of user-level exceptional control flow

Nonlocal Jumps

```
#include <setjump.h>
```

```
int setjmp(jmp_buf env) ;
```

Returns: 0 from setjmp,
nonzero from long jumps

```
#include <setjmp.h>
```

```
void longjmp(jmp_buf env, int retval);
```

Never returns

```
9
10 int main()
11 {
12
13     switch (setjmp(buf)) {
14     case 0:
15         foo(); break;
16     case 1:
17         printf("Detected an error1 condition in foo\n");
18         break;
19     case 2:
20         printf("Detected an error2 condition in bar\n");
21         break;
22     default:
23         printf("Unknown error condition in foo\n");
24     }
25     exit(0);
26 }
```

```
1 #include "csapp.h"
2
3 jmp_buf buf;
4
5 int error1 = 0;
6 int error2 = 1;
7
8 void foo(void),
9     bar(void);
```

```
25
26 /* deeply nested function foo */
27 void foo(void)
28 {
29     if (error1)
30         longjmp(buf, 1) ;
31     bar() ;
32 }
33
34 void bar(void)
35 {
36     if (error2)
37         longjmp(buf, 2) ;
38 }
```


Nonlocal Jumps

- Nonlocal jumps are provided by
 - the `setjmp` function
 - saves the current calling environment in the `env` buffer, for later use by `longjmp`, returns 0
 - includes PC, stack pointer, general-purpose registers
 - the `longjmp` function
 - restores the execution from the `env` buffer
 - triggers a return from the most recent `setjmp` call that initialized `env`
 - the `setjmp` then returns with the nonzero return value `retval`.

Nonlocal Jumps

```
#include <setjump.h>
```

```
int sigsetjmp(sigjmp_buf env, int savemask);
```

Returns: 0 from sigsetjmp,
nonzero from siglongjmps

```
#include <setjmp.h>
```

```
void siglongjmp(sigjmp_buf env, int retval);
```

Never returns

If savemask is nonzero

the process's current signal mask is saved in env
and will be restored if a siglongjmp is later performed
with this env.

```
1  #include "csapp.h"
2  sigjmp_buf buf;
3
4  void handler(int sig) { siglongjmp(buf, 1); }
5
6  int main()
7  {
8      if (!sigsetjmp(buf, 1)) {
9          Signal(SIGINT, handler);
10         Sio_puts("starting\n");
11     }
12     else
13         Sio_puts("restarting\n");
14
15     while(1) {
16         Sleep(1);
17         printf("processing...\n");
18     }
19     exit(0);
20 }
```

A diagram with red arrows illustrates the control flow. One arrow starts from the `siglongjmp` call in the `handler` function (line 4) and points to the `sigsetjmp` call in the `if` statement (line 8). Another arrow starts from the `while` loop (lines 15-18) and points back to the `sigsetjmp` call (line 8), indicating a loop restart triggered by a signal.